



UNIVERSITÄT STUTTGART

ADVANCED SOFTWARE ENGINEERING

Übungsblatt 5

Maximilian Peresunchak (st152466@stud.uni-stuttgart.de)

Nico Reng (st188620@stud.uni-stuttgart.de)

Viorel Tsigos (st188085@stud.uni-stuttgart.de)

Philip Reimann (st182312@stud.uni-stuttgart.de)

Christian Keller (st166512@stud.uni-stuttgart.de)

Johannes Heugel (st183360@stud.uni-stuttgart.de)

Benedikt Wachmer (st177118@stud.uni-stuttgart.de)

Miles Holl (st180549@stud.uni-stuttgart.de)

Wintersemester 2025

19. Januar 2026

Aufgabe 1: Debugging, Slicing und Tarantula I

- (a) i. **Hypothese:** Das Fehlverhalten tritt auf, da in Zeile 6 `lastAdded` auf den Wert $i + 1$ gesetzt wird.

Das bedeutet semantisch, dass der Wert i bereits aufaddiert wurde, dem ist jedoch nicht der Fall. Somit wird der Wert in `lastAdded` immer um 1 zu hoch gespeichert, was dazu führt, dass statt der Summe der Zahlen von 1 bis n die Summe der Zahlen von 2 bis $n + 1$ berechnet wird.

ii. **Vorhersagen:**

1. `summe(10)` sollte laut der Hypothese den Wert 65 zurückgeben.
2. `summe(100)` sollte laut der Hypothese den Wert 5150 zurückgeben.

- (b) In Listing 1 ist der Backward Slicing der Methode `summe` für die Ausgabe in Zeile 11 dargestellt. Dabei wurden alle Anweisungen durchgestrichen, die keinen Einfluss auf den Wert von `sum` in Zeile 11 haben.

```
1  int summe(int n) {  
2      int sum = 0;  
3      int i = 1;  
4      int lastAdded = 0;  
5      while (i <= n) {  
6          lastAdded = i + 1;  
7          sum = sum + lastAdded;  
8          i = i + 1;  
9      }  
10     System.out.println(lastAdded);  
11     return sum;  
12 }
```

Listing 1: Backward Slicing für Zeile 11

Somit zeigt sich, dass nur Zeile 10 keinen Einfluss auf den Wert von `sum` hat. Alle anderen Anweisungen führen über lang oder kurz zu einer Änderung. Beispielsweise sorgt eine Änderung der Abbruchbedingung für die `while`-Schleife in Zeile 5 dafür, dass die Schleife mehr oder weniger oft durchlaufen wird, was wiederum zu mehr oder weniger Additionen von `sum` mit `lastAdded` in Zeile 7 führt. Daher sind auch Zeilen, die `i` oder `lastAdded` ändern, relevant für den Wert von `sum`.

- (c) Um die suspiciousness-Werte mit Tarantula zu berechnen, verwenden wir die folgende Formel:

$$\text{suspiciousness}(s) = \frac{\frac{\text{failed}(s)}{\text{total failed}}}{\frac{\text{passed}(s)}{\text{total passed}} + \frac{\text{failed}(s)}{\text{total failed}}}$$

Für die Zeilen 6, 7 und 10 ergeben sich somit die in Tabelle 1 dargestellten Werte:

Eingabe	T_1	T_2	T_3	T_4	T_5	Suspiciousness
Zeile 5	•	•	•	•	•	
Zeile 6		•	•		•	0.75
Zeile 7		•	•		•	0.75
Zeile 8		•	•		•	
Zeile 10	•	•	•	•	•	0.5
Test-Ergebnis	Pass	Fail	Fail	Pass	Pass	

Tabelle 1: Tarantula Tabelle für die Berechnung der suspiciousness-Werte

Die folgenden Rechnungen zeigen, wie man zu diesem Ergebnis kommt:

$$\begin{aligned}\text{suspiciousness}(6) &= \frac{\frac{2}{2}}{\frac{1}{3} + \frac{2}{2}} = \frac{1}{\frac{1}{3} + 1} = \frac{3}{4} = 0.75 \\ \text{suspiciousness}(7) &= \frac{\frac{2}{2}}{\frac{1}{3} + \frac{2}{2}} = \frac{1}{\frac{1}{3} + 1} = \frac{3}{4} = 0.75 \\ \text{suspiciousness}(10) &= \frac{\frac{2}{2}}{\frac{3}{3} + \frac{2}{2}} = \frac{1}{1 + 1} = \frac{1}{2} = 0.5\end{aligned}$$

Die Werte zeigen, dass die Zeilen 6 und 7 am verdächtigsten sind, was auch mit unserer Hypothese aus Teilaufgabe (a) übereinstimmt. Dadurch, dass der Wert von `lastAdded`, der in Zeile 6 gesetzt wird, in Zeile 7 weiterverwendet wird, ist auch plausibel, dass beide Zeilen den gleichen (hohen) suspiciousness-Wert haben.

Aufgabe 2: Tarantula II

- (a) In Tabelle 2 sind die suspiciousness-Werte für die Zeilen 1-17 dargestellt. Die folgenden Rechnungen zeigen, wie man zu diesen Ergebnissen kommt, wobei äquivalente Zeilen zusammengefasst wurden.

Hinweis: "suspiciousness" wird im Folgenden mit "sus" abgekürzt.

$$\begin{aligned}\text{sus}(1) = \text{sus}(2) &= \frac{\frac{2}{2}}{\frac{3}{3} + \frac{2}{2}} = \frac{1}{1 + 1} = 0.5 \\ \text{sus}(3) &= \frac{\frac{1}{2}}{\frac{0}{3} + \frac{1}{2}} = \frac{0.5}{0 + 0.5} = 1 \\ \text{sus}(4) &= \frac{\frac{1}{2}}{\frac{3}{3} + \frac{1}{2}} = \frac{0.5}{1 + 0.5} = \frac{1}{3} \approx 0.33 \\ \text{sus}(5) &= \frac{\frac{0}{2}}{\frac{1}{3} + \frac{0}{2}} = 0\end{aligned}$$

$$\begin{aligned} \text{sus}(6) = \text{sus}(7) = \text{sus}(8) = \text{sus}(12) = \text{sus}(13) = \text{sus}(17) &= \frac{\frac{1}{2}}{\frac{2}{3} + \frac{1}{2}} = \frac{\frac{1}{2}}{\frac{7}{6}} = \frac{3}{7} \approx 0.43 \\ \text{sus}(9) = \text{sus}(14) &= \frac{\frac{0}{2}}{\frac{2}{3} + \frac{0}{2}} = 0 \\ \text{sus}(10) = \text{sus}(11) = \text{sus}(15) = \text{sus}(16) &= \frac{\frac{1}{2}}{\frac{0}{3} + \frac{1}{2}} = 1 \end{aligned}$$

Nr.	T_1 (Fail)	T_2 (Pass)	T_3 (Pass)	T_4 (Fail)	T_5 (Pass)	Suspiciousness
1	•	•	•	•	•	0.5
2	•	•	•	•	•	0.5
3	•					1
4		•	•	•	•	0.33
5		•				0
6			•	•	•	0.43
7			•	•	•	0.43
8			•	•	•	0.43
9			•		•	0
10				•		1
11				•		1
12			•	•	•	0.43
13			•	•	•	0.43
14			•		•	0
15				•		1
16				•		1
17			•	•	•	0.43

Tabelle 2: Tarantula Tabelle für die Berechnung der suspiciousness-Werte

- (b) Die Gründe für die fehlschlagenden Testfälle liegen mit hoher Wahrscheinlichkeit in den Code-Zeilen mit den größten suspiciousness-Werten, also den Zeilen 3, 10, 11, 15 und 16.
- (c) Beispiele für Code-Zeilen mit Verdächtigkeitswert 0 sind Zeilen 5 und 9. Da dieser Wert bedeutet, dass die genannten Zeilen in keinem fehlschlagenden Testfall ausgeführt wurden, ist die Schlussfolgerung von Tarantula, dass diese Zeilen nicht die Ursache für das Fehlverhalten sein können. Man kann also davon ausgehen, dass diese Zeilen korrekt sind und kann sie deshalb von der Fehlerlokalisierung ausschließen.
- (d) Tarantula basiert auf der Heuristik, dass Codezeilen, die häufig in erfolgreichen Tests (“Pass”) ausgeführt werden, eine geringere Wahrscheinlichkeit haben, fehlerhaft zu sein. Die Formel erhöht den Nenner, wenn **passed(s)** steigt, was den Suspiciousness-Wert senkt.

Wenn eine fehlerhafte Zeile ausgeführt wird, der Test aber trotzdem besteht (weil der Fehler sich bei den spezifischen Eingabedaten nicht auf das Endergebnis auswirkt), interpretiert Tarantula dies als Indiz für die Korrektheit der Zeile. Dies verfälscht das Ergebnis und senkt den Verdachtswert der eigentlich fehlerhaften Zeile irrtümlich.

Ein Beispiel hierfür wäre die folgende Situation: In einer Methode soll $x + x$ berechnet werden, aber es wurde fälschlicherweise $x * x$ implementiert. Angenommen, es gibt einen Testfall mit der Eingabe $x = 2$, der das korrekte Ergebnis 4 erwartet. Trotz des Fehlers in der Implementierung liefert die Methode $2 * 2 = 4$, was zum Bestehen des Tests führt. Tarantula würde diese fehlerhafte Zeile als weniger verdächtig einstufen, da sie in einem erfolgreichen Test ausgeführt wurde, obwohl sie tatsächlich fehlerhaft ist.